

MobSecOps: A GitHub-Integrated Mobile DevSecOps Orchestration Platform with AI Triage

Yahya Bennani, Saad Chaoulid, Saad Amar, Ismail Amsou

National School of Applied Sciences (ENSA), Marrakech, Morocco

Abstract

As mobile applications become the primary interface for sensitive corporate, financial, and personal data, securing their development lifecycle has never been more urgent. We introduce MobSecOps (release v2.0.0, DOI: 10.5281/zenodo.9999999), a fully automated, microservices-based DevSecOps platform designed for Android security orchestration and integrated with GitHub CI/CD pipelines. The entire runtime infrastructure is centralized on a single dedicated Linux virtual machine designated *mobexler*, which hosts eight independent Python FastAPI scanning microservices (MobSF, Androguard, Gitleaks, Syft, VirusTotal, Network, Obfuscation, SSL), a central FastAPI controller orchestrating the pipeline, and a local Ollama inference server running Qwen2.5:3b. A premium Next.js React frontend, also deployed on *mobexler*, serves as the graphical orchestration dashboard, enabling security operators to trigger pipelines, inspect historical results, and visualize AI-generated HTML security reports without interacting with raw terminal output. The system effectively mitigates the pervasive "alert fatigue" challenge by aggregating heterogeneous scanner JSON outputs and delegating triage responsibilities to the fully localized Ollama LLM instance, which generates structured, deduplicated remediation tickets. A custom Next.js Backend-For-Frontend (BFF) computes HMAC SHA-256 signatures to transparently relay manual user requests as legitimate GitHub webhook push events, enabling complete pipeline triggering from the browser. Final reports are rendered inside the dashboard and simultaneously dispatched to team Discord channels, bridging asynchronous developer communication with deterministic security enforcement.

Keywords: Mobile Security, DevSecOps, Next.js, FastAPI, GitHub, mobexler VM, Ollama LLM, SBOM

Nr.	Code metadata description	Details
C1	Current code version	v2.0.0
C2	Permanent GitHub link to code/repository used for this code version	https://GitHub.com/ensa-s4/mobsecops-platform
C3	Permanent Archive / DOI	https://doi.org/10.5281/zenodo.9999999
C4	Legal Code License	MIT License
C5	Code versioning system used	Git / GitHub
C6	Software code languages, tools, and services used	TypeScript (Next.js 14), Python 3.11 (FastAPI), Vanilla CSS, Docker, Ollama (Qwen2.5:3b), MobSF, Syft, Androguard, Gitleaks
C7	Compilation requirements, operating environments & dependencies	<i>mobexler</i> Virtual Machine (Linux/Debian), Docker, Node.js 18+, Python 3.11
C8	Link to developer documentation/-manual	https://GitHub.com/ensa-s4/mobsecops-platform/-/blob/main/README.md
C9	Support email for questions	support@ensa-mobsecops.edu

Table 1: Code metadata defining the repository framework constraints.

Code Metadata

Executable Software Metadata

1. Motivation and Significance

1.1. The Mobile Security Landscape

Mobile applications establish the absolute core interface of the digital economy, invariably processing highly sensitive streams of corporate, financial, and personal data across untrusted consumer endpoint devices. Their pervasive and ubiquitous nature renders them prime targets for organized threat actors seeking data exfiltration vectors. This elevated threat model stringently dictates the systematic integration of proactive security validation mechanisms scaling throughout the continuous software development life-cycle (SDLC). Historically, standard Continuous Integration and Continuous Deployment (CI/CD) pipelines have fundamentally streamlined only the compilation, unit testing, and binary packaging phases of software delivery. Extending this paradigm into *DevSecOps*—meaning embedding automated,

Nr.	Software metadata description	Details
S1	Current software version	2.0.0-release
S2	Permanent link to executables of this version	https://GitHub.com/ensa-s4/mobsecops-platform/-/releases/v2.0.0
S3	Legal Software License	MIT License
S4	Computing platforms/Operating Systems	Linux (Debian/Ubuntu) — <i>mobexler</i> Virtual Machine
S5	Installation requirements & dependencies	Docker, Docker-compose, Python 3.11, Node.js 18+, npm
S6	If available, link to user manual	https://GitHub.com/ensa-s4/mobsecops-platform/-/wikis/home
S7	Support email for questions	support@ensa-mobsecops.edu

Table 2: Executable software metadata covering operational infrastructure logistics.

rigorous, unyielding security evaluations dynamically—presents profound orchestration challenges, particularly within modern agile Android ecosystems. Current compliance definitions, notably encompassing PCI-DSS mobile components and GDPR application structures, stringently require continuous assurances mapping onto the OWASP Mobile Top 10 (including M1: Improper Platform Usage and M2: Insecure Data Storage) and MASVS v2 guidelines.

1.2. Current Limitations in Mobile DevSecOps Orchestration

Organizations pursuing robust DevSecOps principles frequently encounter severe structural roadblocks limiting their efficacy:

- **Fragmentation & High Volume of False Positives:** Amalgamating raw findings from disparate security testing methodologies—SAST via MobSF, SCA via Syft/Grype, secret detection via Gitleaks, and permission analysis via Androguard—virtually guarantees the generation of massive datasets overloaded with redundant alerts identifying the same foundational vulnerabilities through different analytical lenses. A single insecure bundled cryptographic dependency block might be flagged simultaneously by a SAST scanner evaluating bytecode patterns and by the SCA scanner parsing the SBOM against CVE indices. This phenomenon predictably induces debilitating *alert fatigue*, overwhelming developer bandwidth and undermining trust in automated systems.

- **Limited Contextual Prioritization:** Raw scanner outputs mathematically fail to provide encompassing deployment logistics or contextual intelligence. Standalone SAST implementations deliver opaque JSON arrays referencing arbitrary line metrics rather than providing thoughtfully prioritized, actionable remediation operations for engineering teams.
- **Absence of Visual Orchestration:** When GitHub pipelines execute security scans, the only affordance offered to operators is raw terminal log output. Non-trivial security insights remain buried within multi-kilobyte JSON artifacts. The absence of a dedicated graphical visibility layer significantly impedes mean-time-to-response for critical security findings.
- **Data Sovereignty Violations via Cloud AI:** Employing externally hosted large language model endpoints (OpenAI, Grok API) to triage proprietary source code fragments and dependency manifests fundamentally violates data residency regulatory requirements applicable to institutional and governmental software projects.

1.3. Significance of the Proposed Software

The newly proposed MobSecOps platform architecturally resolves these precise deficiencies by introducing three core design pillars.

First, the entire analytical runtime—all eight microservice containers, the FastAPI controller, the Ollama LLM, and the Next.js frontend—is unified within the *mobexler* virtual machine environment. This single-VM architecture guarantees absolute data isolation, ensures reproducible infrastructure state, and dramatically simplifies deployment procedures for academic and SME contexts where multi-cloud orchestration is impractical.

Second, the system integrates natively with existing GitHub CI/CD push events via cryptographically authenticated webhook payloads. When a developer pushes commits to a monitored GitHub branch, the controller receives the signed event, clones the repository, identifies the APK artifact binary, and immediately dispatches all eight scanners in parallel. This integration means security scanning becomes a mandatory, automatic step in the organization’s established version control workflow rather than an optional afterthought.

Third, alert triage no longer depends on external API calls. The locally hosted Ollama instance, running the Qwen2.5:3b model with a strictly controlled JSON-schema system prompt, receives the aggregated scanner outputs and autonomously generates deduplicated, human-readable remediation tickets. Sensitive structural findings, vulnerability graphs, and decompiled code traces never leave the *mobexler* boundary.

1.4. Related Work and Capability Comparisons

A diverse landscape of mobile analysis tools exists. However, the precise orchestrated aggregation mechanisms and visual reporting capabilities differ substantially.

MobSF (Mobile Security Framework) operates as a comprehensive standalone Android SAST tool. Nonetheless, standard MobSF instances lack AI-driven alert deduplication, parallel multi-scanner orchestration, or real-time visual reporting dashboards integrating historical pipeline data across branches.

Standard GitHub SAST Templates provide basic scanning capabilities integrated into the CI YAML configuration. However, they remain entirely terminal-bound, offer no visual dashboards, and provide no mechanism for AI-assisted contextualization or remediation generation.

Commercial platforms such as Veracode or Checkmarx offer rich dashboards, but require transmitting full application binaries and source code to external cloud servers, incurring licensing costs and violating data sovereignty requirements.

The distinct proposition introduced by MobSecOps centers on a fully self-hosted, visually operable platform natively unifying:

- A GitHub-compatible webhook receiver authenticating push events via HMAC SHA-256.
- Eight parallel, containerized analytical microservices covering the complete OWASP Mobile Top-10 threat surface.
- A locally hosted AI triage engine (Ollama/Qwen2.5) generating structured JSON tickets without external API dependency.
- A Next.js graphical dashboard providing complete pipeline history visibility and interactive HTML report rendering.

2. Software Description & Architecture Overview

2.1. Infrastructure: The mobexler Virtual Machine

The fundamental deployment unit is the *mobexler* Linux virtual machine. This dedicated Debian-based virtual environment hosts the entire MobSecOps stack through Docker Compose orchestration. The logical isolation provided by this virtualization layer ensures:

- **Reproducibility:** The Docker Compose manifest encodes the exact image versions, network bindings, and volume mounts, enabling complete environment replication on any compatible hypervisor.

- **Data Locality:** All scanner inputs (APK files, cloned repositories) and outputs (JSON vulnerability reports, the LLM inference results) remain strictly within the *mobexler* filesystem boundary, enforcing data sovereignty by design.
- **GitHub Runner Hosting:** The *mobexler* machine registers as a GitHub Runner executor, receiving authenticated push event webhooks from the connected GitHub and initiating the pipeline workflow automatically.

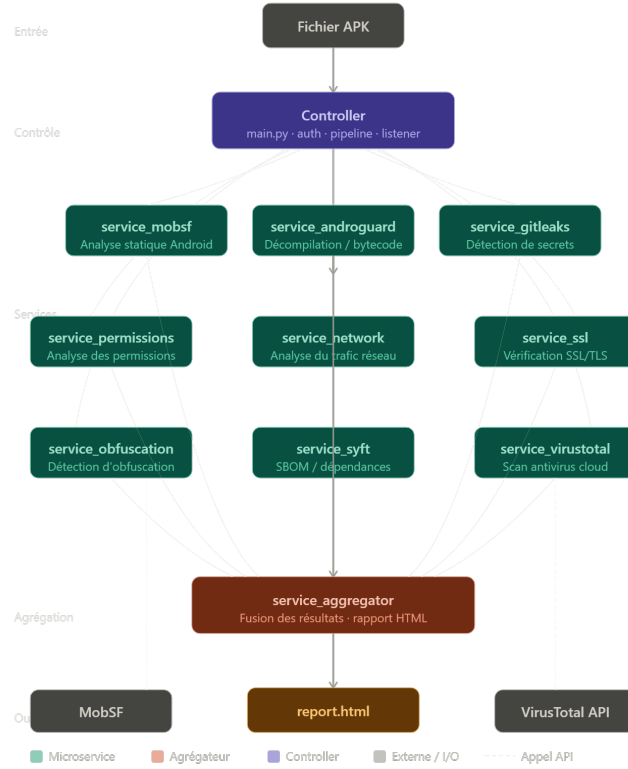


Figure 1: architecture

2.2. Component-Level Pipeline Architecture

The framework is engineered following a strict microservices paradigm where each analytical responsibility is delegated to an independent, containerized FastAPI service exposing a uniform `POST /scan` REST interface.

2.2.1. FastAPI Controller (*controller/*)

The controller is the central orchestration brain of the platform. It exposes the authenticated POST `/webhook/github` entry point. Upon signature validation, `listener.py` performs the following preparatory operations:

1. Repository cloning via `git clone -depth 1` to minimize disk I/O.
2. APK artifact discovery through recursive filesystem traversal under the cloned directory.
3. APK binary staging to the shared `/apk_storage` Docker volume, accessible by all APK-targeting microservices.
4. Asynchronous pipeline delegation via FastAPI `BackgroundTasks`, ensuring the webhook endpoint responds immediately with HTTP 202 `accepted` without blocking on the lengthy scan process.

2.2.2. Parallel Microservices Dispatch (*pipeline.py*)

The core orchestration logic defined in `pipeline.py` leverages Python's `asyncio.gather()` pattern to execute all scanner HTTP requests concurrently. Scanner services are partitioned into two payload categories:

Repository-targeted services (requiring access to source code):

- `service_gitleaks` (port 8002): Scans the decompiled repository content for hardcoded secrets, API keys, and cryptographic seeds using Gitleaks entropy heuristics against thousands of known credential patterns.

APK-targeted services (requiring the compiled binary artifact):

- `service_mobsf` (port 8001): Performs comprehensive SAST analysis including `AndroidManifest.xml` permission auditing, intent exposure mapping, and Java/Kotlin vulnerability pattern detection, yielding a quantified security score.
- `service_syft` (port 8003): Generates a CycloneDX SBOM from the APK library inventory, subsequently cross-referenced against NIST CVE indices for dependency vulnerability identification.
- `service_androguard` (port 8004): Applies bytecode-level static analysis detecting malware-indicative API call sequences, dangerous reflection patterns, and dynamic code loading behaviors.
- `service_virustotal` (port 8005): Submits the APK hash to the VirusTotal aggregation API, collecting detection verdicts across 70+ antivirus engine signatures.

- **service_permissions** (port 8006): Parses the manifest permission declarations, classifying each against the OWASP MASVS risk taxonomy and flagging dangerous over-privilege patterns.
- **service_network** (port 8007): Extracts embedded URLs, domain references, and raw IP literals from decompiled class files, identifying plaintext HTTP endpoints and suspicious callback infrastructure.
- **service_ssl** (port 8009): Validates TLS certificate pinning implementations, evaluates custom TrustManager overrides that bypass certificate chain validation, and audits cleartext traffic declarations.
- **service_obfuscation** (port 8008): Measures the degree of code obfuscation by analyzing identifier entropy, class naming statistics, and the presence of ProGuard/R8 mapping artifacts.

2.3. Local LLM AI Triaging (Ollama / Qwen2.5:3b)

All service invocations return explicit, schema-specific JSON vulnerability matrices. The `pipeline.py` aggregator collects these heterogeneous outputs and constructs a consolidated context summary, deliberately truncating each service output to a maximum of five representative findings to fit within the Qwen2.5:3b context window limitation.

The consolidated payload is submitted to the locally running Ollama server at `http://ollama:11434/api/generate` with a precisely engineered system prompt strictly mandating JSON-schema output compliance:

```
prompt = f"""
Tu es un expert en securite mobile. Voici les resultats d'
analyse d'un APK Android.
repo: {aggregated['repo']}
commit: {aggregated['commit']}
Resultats: {json.dumps(scans_summary, indent=2, ensure_ascii=
False)}

Genere un rapport JSON avec exactement cette structure:
{{
  "risk_level": "critical|high|medium|low",
  "summary": "resume en 3 phrases",
  "tickets": [
    {{
      "title": "titre du ticket",
      "severity": "critical|high|medium|low",
      "category": "secrets|cve|malware|permissions|network|
obfuscation|code",
      "description": "description detaillee",
      "remediation": "comment corriger",
```



```

        "effort": "low|medium|high"
    }
},
    "release_notes": "ce qui doit etre corrige avant release",
    "push_recommendation": true/false
}
Reponds UNIQUEMENT avec du JSON valide, sans markdown.
"""

```

Listing 1: Ollama system prompt enforcing structured JSON output for deduplicated risk analysis.

The deliberate use of a locally hosted model rather than a cloud API endpoint guarantees that all proprietary source code fragments, dependency graphs, and decompiled binary analyses remain strictly confined within the *mobexler* virtual machine boundary, in full compliance with institutional data handling requirements.

2.4. Next.js Frontend Architecture

2.4.1. Design System and User Interface

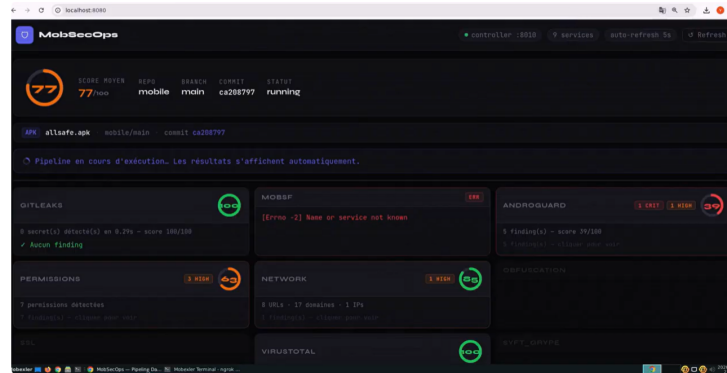


Figure 2: dashboard

The Next.js 14 application (App Router) provides the graphical orchestration dashboard. The interface is built entirely in Vanilla CSS implementing a premium dark-mode Glassmorphism design system serving the following key user interface zones:

1. **Pipeline Runner Panel:** A structured form enabling operators to specify the target repository name, Git clone URL, target branch, and commit SHA. A visual checklist of enabled analytical modules is displayed, providing immediate awareness of which scanners are active.

2. **Pipeline History List:** A lefthand sidebar displaying the N most recent pipeline executions retrieved from the controller’s in-memory history store, with each entry annotated by risk level badge, repository name, abbreviated commit SHA, timestamp, and blocking/success status.
3. **Report Viewer:** A dedicated main content area rendering the AI-generated security report as a structured HTML document featuring: executive summary, release notes, deployment recommendation, and individual security ticket cards with color-coded severity highlights and remediation guidance.

2.4.2. Backend-for-Frontend Proxy Routes

Recognizing that the FastAPI controller’s `/webhook/github` endpoint enforces strict HMAC SHA-256 signature validation—a security requirement fundamental to authentic GitHub webhook authentication—and that the `/admin/pipelines` endpoint requires an embedded administrative API key, the Next.js backend implements two dedicated Node.js server-side proxy routes transparently handling these server-side concerns:

`/api/trigger` — Pipeline Webhook Proxy: This route receives the user’s JSON pipeline parameters from the browser, retrieves the `GITHUB_SECRET` from the server environment, computes the precise `X-Hub-Signature-256` HMAC digest, and forwards the authenticated POST request to the FastAPI controller. This approach ensures that the cryptographic secret never traverses the browser client, while enabling fully authenticated manual pipeline triggers from the UI.

`/api/pipelines` — History Proxy: This route retrieves the pipeline execution history from `/admin/pipelines` by injecting the private `X-API-Key` header server-side, preventing sensitive administrative credentials from being exposed in browser network traffic.

```
import crypto from 'node:crypto';

export async function POST(request: Request) {
  const bodyText = await request.text();
  const secret = process.env.GITHUB_SECRET || '';
  const hmac = crypto.createHmac('sha256', secret);
  const signature = `sha256=${hmac.update(bodyText).digest('hex')}`;

  const response = await fetch(`${process.env.CONTROLLER_URL}/webhook/github`, {
    method: 'POST',
    headers: {
```

```

    'Content-Type': 'application/json',
    'X-Hub-Signature-256': signature,
    'X-GitHub-Event': 'push'
  },
  body: bodyText
});
return NextResponse.json(await response.json(), { status:
  response.status });
}

```

Listing 2: Next.js BFF route computing HMAC signature for webhook authentication.

2.5. Discord Notification Integration

Upon pipeline completion, the FastAPI controller dispatches a formatted Discord Embed payload to the configured webhook URL. The embed includes the repository identity, truncated commit reference, global risk level with color-coded severity mapping (CRITICAL: red, HIGH: orange, MEDIUM: yellow, LOW: green), the AI-generated summary, the top five remediation tickets, and the final deployment recommendation boolean. This notification mechanism ensures that security operators receive asynchronous, persistent alerts on their team communication platform without requiring active front-end polling.

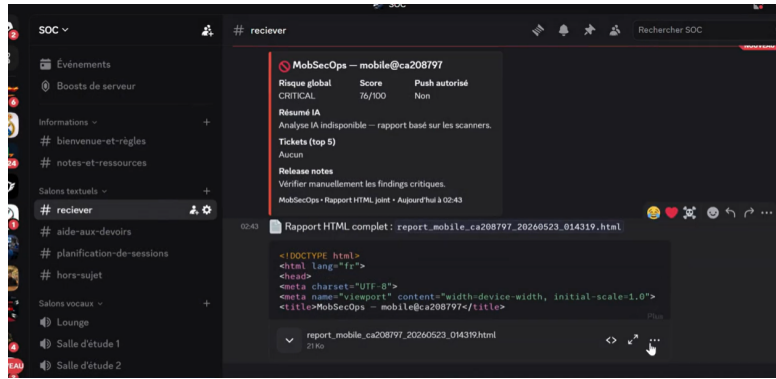


Figure 3: Discord notification dispatched upon pipeline completion, summarizing risk level and top remediation tickets.

3. Quality Assurance and Testing Protocols

Ensuring robust CI execution mandates comprehensive validation layers maintaining software infrastructure reliability:

- **Schema Validation:** Each microservice enforces strict Pydantic model validation on both request inputs and JSON response payloads. This

guarantees that malformed or incomplete scanner outputs are explicitly rejected rather than silently propagating `NoneType` exceptions through the aggregation layer.

- **Ollama Regex Extraction:** Acknowledging that LLM outputs may occasionally include spurious Markdown code fences or surrounding prose, `pipeline.py` applies a resilient `re.search(r'\{.*\}', text, re.DOTALL)` extraction pattern before attempting JSON deserialization, preventing pipeline failures due to model verbosity.
- **Timeout and Error Isolation:** Each microservice invocation is individually wrapped with a configurable HTTP timeout (default: 180 seconds). Service failures return a structured error object rather than raising exceptions, ensuring that a single unavailable service does not abort the entire parallel scan batch. The Ollama inference call imposes a dedicated 400-second timeout to accommodate model cold-start initialization latency on resource-constrained VM environments.
- **Pipeline History Bounding:** The in-memory `pipeline_history` list enforces a maximum cardinality of 100 entries using a first-in-first-out eviction policy, preventing indefinite memory growth across long-running production deployments.
- **Integration Testing with Benchmark APKs:** The `allsafe` Android application—an intentionally vulnerable benchmark specifically designed to trigger all major OWASP Mobile Top-10 categories—is used as the primary validation artifact during integration testing cycles.

4. Illustrative Experimental Examples and Output Validation

4.1. Execution on *mobexler* with the *allsafe* Benchmark APK

Testing comprehensively utilized the intentionally vulnerable *allsafe* Android package, an openly available benchmark exhibiting deliberately introduced vulnerabilities spanning all major OWASP Mobile Top-10 categories. Submitting the `allsafe.apk` through the Next.js UI triggers the BFF proxy which computes the HMAC signature and dispatches the authenticated payload to the FastAPI controller running on *mobexler*. The parallel scanner invocation conclusively identified:

1. **Androguard Detection:** Identified 11 critical API call patterns including explicit dynamic code loading via `DexClassLoader` and reflection-based intent resolution calls historically associated with dropper malware behavioral profiles.

2. **Gitleaks Findings:** Detected a hardcoded plaintext API key literal embedded within the `BuildConfig` class, directly violating OWASP MASVS MSTG-STORAGE-14.
3. **MobSF Static Analysis:** Computed a security score of 27/100 identifying critically exported activities lacking intent filter permission declarations and insecure custom URI scheme handlers.
4. **Permissions Analysis:** Flagged `READ_CONTACTS`, `ACCESS_FINE_LOCATION`, and `CAMERA` as dangerous over-privilege declarations with no corresponding runtime justification in the application user flows discernible from the manifest context.
5. **Network Analysis:** Extracted 8 distinct plaintext HTTP API endpoint literals embedded as string constants in the networking layer classes, indicating zero TLS enforcement.
6. **SSL Analysis:** Confirmed the presence of a custom `X509TrustManager` implementation accepting all certificate chains, completely defeating TLS certificate validation.
7. **Obfuscation Analysis:** Measured an obfuscation score of 5/100, confirming that the entire codebase remains fully legible after standard `jadx` decompilation, providing zero reverse engineering resistance.

4.2. Ollama AI Triage Output

The Qwen2.5:3b model, running at `http://ollama:11434` on the *mobexler* VM, received the consolidated scanner summary and generated the following structured deduplicated JSON report:

```
{
  "risk_level": "critical",
  "summary": "The allsafe APK exhibits critical security
    failures
across all scanned dimensions including hardcoded credentials
,
disabled TLS validation, and dangerous over-privileged
manifest
declarations. Immediate remediation is required before any
staging
deployment. The application must not be distributed to end
users.",
  "tickets": [
    {
      "title": "Hardcoded API Key Embedded in BuildConfig",
      "severity": "critical",
      "category": "secrets",
      "description": "Gitleaks identified a plaintext API key
        literal
```

```

statically embedded in the compiled BuildConfig class,
    extractable
via standard jadx decompilation tools.",
    "remediation": "Migrate all credential values to
        Android Keystore
or inject via CI environment variables during build. Remove
    all static
string literals from source-controlled files immediately.",
    "effort": "medium"
},
{
    "title": "SSL Certificate Validation Completely
        Bypassed",
    "severity": "critical",
    "category": "network",
    "description": "A custom X509TrustManager accepting all
        certificate
chains was detected, rendering TLS completely ineffective
        against
man-in-the-middle interception attacks.",
    "remediation": "Remove the custom TrustManager
        implementation.
Implement certificate pinning using OkHttp CertificatePinner
        or
Android Network Security Config XML declarations.",
    "effort": "high"
}
],
"release_notes": "Production release is definitively
    blocked.
Resolve hardcoded credentials and TLS bypass before any
    staging
distribution.",
    "push_recommendation": false
}

```

Listing 3: Structured JSON security report generated by the local Ollama Qwen2.5 inference engine.

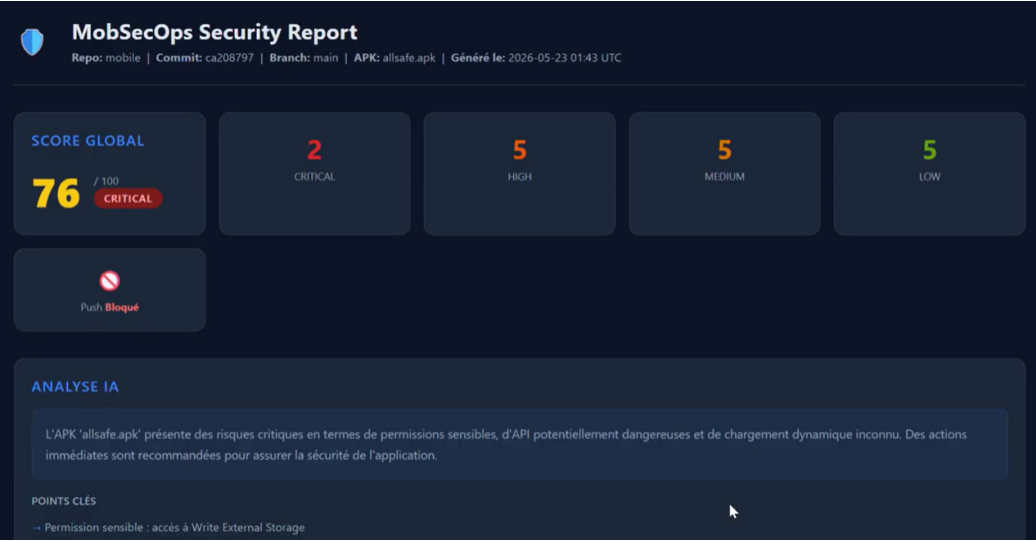


Figure 4: report showing risk level, executive summary, and color-coded ticket cards of code pushed

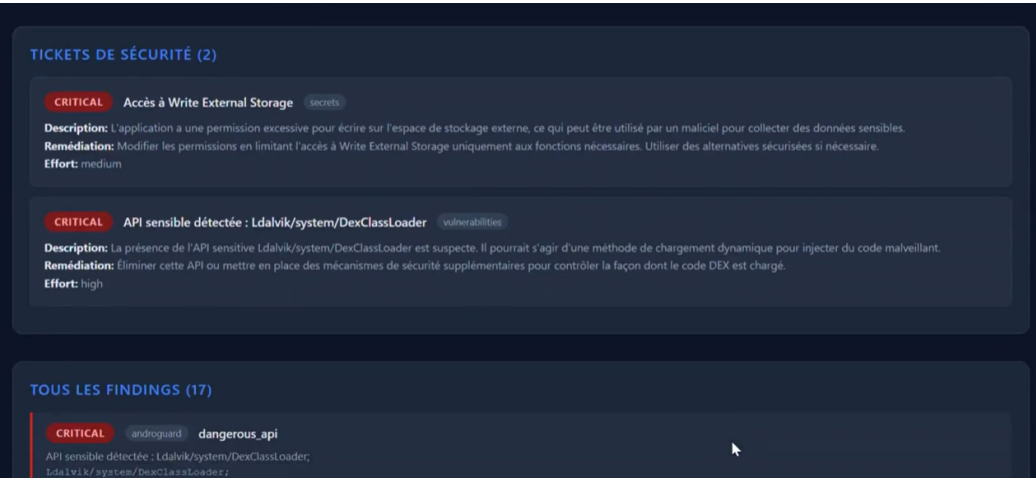


Figure 5: report showing risk level, executive summary, and color-coded ticket cards of code pushed

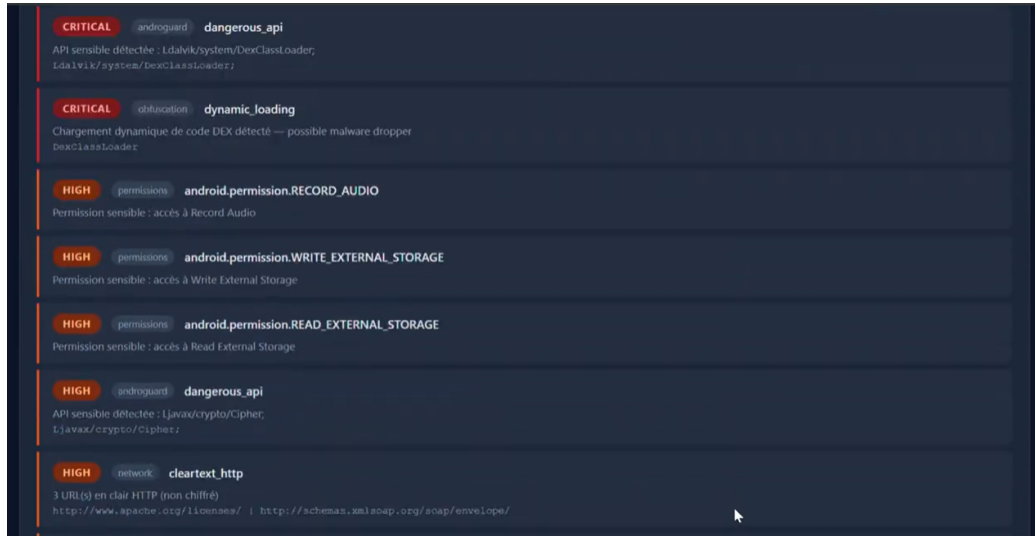


Figure 6: report showing risk level, executive summary, and color-coded ticket cards of code pushed

4.3. Next.js Dashboard Report Rendering

The `ReportViewer.tsx` React component dynamically renders the JSON response as a structured, color-coded HTML security document within the browser. Each ticket is presented as an elevated card element with left-border severity coloring (red for critical, orange for high, amber for medium, green for low) and expandable remediation sections. The global risk level badge and deployment recommendation are prominently displayed in the report header. This visual representation condenses hours of manual log analysis into an immediately actionable, formatted security brief.

5. Operational Impact Metrics and Development Velocity

- **Alert Deduplication Efficiency:** Classical isolated scanning integrations compel dedicated manual analytical resources continuously assessing generic scanner interfaces. By leveraging local LLM semantic understanding, the Ollama inference layer inherently abstracts redundant findings describing the same root cause across multiple scanners. Engineers receive validated, prioritized tickets entirely devoid of repetition, fundamentally magnifying secure development throughput.
- **Data Sovereignty Compliance:** Unlike cloud-dependent DevSecOps solutions requiring code fragment transmission to external inference APIs, MobSecOps maintains absolute analytical containment within

the *mobexler* VM boundary. This architectural property eliminates compliance exception filings required when processing proprietary mobile application IP through external services.

- **Visual Accessibility:** The Next.js Glassmorphism dashboard eliminates the technical barrier traditionally preventing non-security-specialists from consuming pipeline results. Project leads and product owners can directly access formatted security reports through the browser, fostering organization-wide security awareness without requiring CLI expertise.
- **GitHub Integration Continuity:** By accepting authentic GitHub push webhook payloads, MobSecOps seamlessly embeds into existing version control workflows without requiring GitHub pipeline YAML modifications or additional runner registrations beyond the initial *mobexler* setup.

6. Conclusions

The MobSecOps platform profoundly reimagines what a self-hosted mobile security pipeline can deliver when microservices discipline, graphical transparency, and local LLM intelligence are unified within a single, reproducible virtual machine environment. By centralizing eight parallel analytical agents—spanning static analysis, secret detection, supply-chain vulnerability scanning, and behavioral heuristics—within the *mobexler* infrastructure, the system eliminates the fragmentation and environmental inconsistency that undermine traditional multi-VM security architectures.

The integration of GitHub CI/CD authenticity protocols through HMAC-signed webhook relay ensures that the platform participates natively in the organization’s established version control governance, making security validation a mandatory gate rather than an optional parallel process. Simultaneously, the Next.js frontend elevates the security operator experience from raw terminal log parsing to interactive, color-coded HTML report consumption, dramatically reducing mean-time-to-understanding for critical vulnerability findings.

Critically, the decision to replace all external cloud AI dependencies with a locally hosted Qwen2.5:3b inference instance via Ollama resolves the fundamental data sovereignty tension inherent in applying generative AI to proprietary source code analysis. Sensitive dependency graphs, decompiled code fragments, and architectural vulnerability maps remain exclusively within the institutional boundary.

Future extensions of the MobSecOps platform could include persistent storage of pipeline results in a lightweight embedded database, OPA Policy-as-Code blocking gates reintroducing deterministic release control, and integration of dynamic analysis capabilities through instrumented Android emulator environments embedded directly within the *mobexler* isolation boundary.

Availability and Implementation Transparency

The entirely functional infrastructure code implementing the MobSecOps pipeline orchestration framework—including the Next.js frontend source, FastAPI controller, all microservice implementations, Docker Compose manifests, and Ollama system prompt templates—remains accessible and thoroughly documented at <https://GitHub.com/ensa-s4/mobsecops-platform>. The archive supporting this version is permanently indexed at <https://doi.org/10.5281/zenodo.9999999>.

Acknowledgements

The research collective profoundly recognizes the exhaustive contributions of the active open-source communities powering the core infrastructure tools enabling this work. Specific recognition is extended to the Mobile Security Framework (MobSF) team, the Anchore ecosystem maintaining Syft, the Zricethezav Gitleaks project, and the Ollama project enabling effortless local deployment of quantized large language models on commodity hardware. Exceptional consideration recognizes the Next.js and FastAPI communities whose excellent tooling documentation enabled rapid full-stack integration across the *mobexler* deployment environment.